

S.A.G.E. - Sistema di Analisi e Gestione Economica

Corso: Basi di Dati
Anno accademico 2025/2026

Componenti del gruppo

Antonio Scharmuller
Riccardo Dallacasa

Matricole

Matricola 000000
Matricola 000000

Email universitarie

antonio.scharmuller@studio.unibo.it
riccardo.dallacasa@studio.unibo.it

Docente: Nome Cognome

17 maggio 2026

Indice

1	Introduzione	1
1.1	Obiettivo del progetto	1
1.2	Descrizione generale del sistema	1
1.3	Ambito del sistema	1
1.4	Utenti e ruoli previsti	1
1.5	Tecnologie utilizzate	1
1.6	Scelte progettuali preliminari	1
2	Analisi dei requisiti	2
2.1	Requisiti in linguaggio naturale	2
2.2	Analisi delle ambiguità e decisioni progettuali	2
2.3	Specifiche ristrutturate	2
2.4	Glossario dei termini essenziali	2
2.5	Classi di utenza / viste del sistema	3
2.6	Elenco delle operazioni principali	3
2.7	Vincoli espliciti e impliciti	3
3	Progettazione concettuale	3
3.1	Criteri generali di modellazione	3
3.2	Raffinamenti dello schema	3
3.2.1	Gerarchia delle transazioni (ISA)	3
3.2.2	Modellazione ibrida (Sistema vs Personale)	3
3.2.3	Reificazione del budget	4
3.3	Criticità rilevate	4
3.4	Schema scheletro E/R	5
3.5	Sviluppo delle viste o degli ambiti	5
3.5.1	Vista Studente e portafoglio personale	5
3.5.2	Vista classificazione delle transazioni	5
3.5.3	Vista budget e pianificazione	5
3.5.4	Vista spese ricorrenti e documenti	5
3.5.5	Vista amministratore e cataloghi globali	5
3.6	Raffinamenti proposti	5
3.7	Gerarchie presenti nello schema	5
3.8	Associazioni principali e cardinalità	5
3.9	Vincoli non rappresentabili direttamente nello schema E/R	5
3.10	Integrazione delle viste	5
3.11	Schema E/R finale	5
3.12	Dizionario di entità e associazioni	5
4	Specifiche funzionali	5
4.1	Elenco delle funzionalità richieste	5
4.2	Operazioni di inserimento	5
4.3	Operazioni di modifica	5
4.4	Operazioni di eliminazione	5
4.5	Operazioni di interrogazione	5
4.6	Operazioni statistiche / amministrative	5
4.7	Input, output e tabelle coinvolte per ogni operazione	5

5	Progettazione logica	5
5.1	Stima del volume dei dati	5
5.2	Descrizione delle operazioni principali e frequenza	5
5.3	Schemi di navigazione	5
5.4	Tabelle degli accessi	5
5.5	Analisi delle ridondanze	5
5.6	Raffinamento dello schema	6
5.6.1	Eliminazione delle gerarchie	6
5.6.2	Soluzione logica anticipata	6
5.6.3	Eliminazione degli attributi composti	6
5.6.4	Gestione degli attributi multivalore	6
5.6.5	Scelta delle chiavi primarie	6
5.6.6	Traduzione delle associazioni	6
5.7	Traduzione di entità e associazioni in relazioni	6
5.8	Schema relazionale finale	6
5.9	Vincoli di integrità	6
5.10	Controllo della normalizzazione	6
6	Implementazione SQL	7
6.1	Script di creazione del database	7
6.2	Creazione delle tabelle	7
6.3	Definizione di chiavi primarie e chiavi esterne	7
6.4	Vincoli NOT NULL, UNIQUE, CHECK	7
6.5	Trigger, procedure o viste, se usati	7
6.6	Popolamento iniziale del database	7
6.7	Query SQL delle operazioni principali	7
6.7.1	Problemi e criticità	7
6.7.2	Analisi	7
6.7.3	Soluzione SQL	7
6.7.4	Query di supporto e operazioni amministrative	8
6.7.5	Motivazioni	9
7	Progettazione dell'applicazione	9
7.1	Architettura generale dell'applicazione	9
7.2	Tecnologie usate	10
7.3	Struttura dei package Java	10
7.4	Collegamento al database tramite JDBC	10
7.5	Organizzazione delle classi principali	10
7.6	Gestione degli errori e validazione input	10
7.7	Interfaccia grafica Java Swing	11
7.8	Schermate principali	11
7.9	Flussi d'uso principali	11
8	Test e validazione	11
8.1	Dati di test utilizzati	11
8.2	Test delle query SQL	11
8.3	Test delle funzionalità applicative	11
8.4	Casi limite gestiti	11
8.5	Errori noti o limitazioni	11

9 Conclusioni	11
9.1 Risultato ottenuto	11
9.2 Scelte progettuali principali	11
9.3 Possibili sviluppi futuri	11
A Script SQL completo	12
B Query SQL complete	12
C Screenshot dell'applicazione	12
D Manuale utente sintetico	12
E Eventuale link al repository	12

1 Introduzione

1.1 Obiettivo del progetto

La proposta iniziale di S.A.G.E. è la realizzazione di un sistema per la gestione e l'analisi delle finanze personali universitarie. L'obiettivo è supportare lo studente nella registrazione, organizzazione e analisi delle proprie transazioni economiche, offrendo strumenti per classificazione, pianificazione e monitoraggio nel tempo.

1.2 Descrizione generale del sistema

S.A.G.E. è progettato come un'applicazione multiutente in cui ogni studente dispone di un portafoglio finanziario personale isolato. Il sistema consente la registrazione di spese e entrate, la categorizzazione tramite categorie e tag, la definizione di budget temporali e la gestione di documenti digitali associati alle transazioni.

1.3 Ambito del sistema

L'ambito del progetto comprende:

- gestione individuale del portafoglio economico personale;
- classificazione delle transazioni tramite categorie gerarchiche e tag trasversali;
- gestione dei budget mensili e per categoria;
- supporto per spese ricorrenti come modelli astratti;
- associazione di documenti digitali alle spese;
- analisi e reportistica aggregata.

Non sono previste funzionalità di condivisione di spese o conti comuni tra utenti.

1.4 Utenti e ruoli previsti

Il sistema prevede due ruoli principali:

- **Utente:** gestisce le proprie transazioni, categorie e tag personali, definisce budget, registra documenti e visualizza report finanziari.
- **Amministratore:** gestisce gli elementi globali del sistema (categorie, tag, fonti) e analizza dati aggregati anonimi senza accedere ai dettagli finanziari personali degli utenti.

1.5 Tecnologie utilizzate

DBMS: MySQL

Linguaggio applicativo: Java

Interfaccia: Java Swing

Connessione DB: JDBC

1.6 Scelte progettuali preliminari

Fin dalla fase iniziale sono state adottate alcune scelte progettuali rilevanti: l'isolamento dei dati finanziari tra utenti, la distinzione tra elementi di classificazione globali e personali e la memorizzazione dei documenti digitali tramite percorso file anziché tramite BLOB nel database. Tali decisioni mirano a garantire privacy, personalizzazione e maggiore semplicità di gestione del DBMS.

2 Analisi dei requisiti

2.1 Requisiti in linguaggio naturale

Il sistema deve supportare la registrazione e il monitoraggio dei movimenti finanziari personali di uno studente universitario, distinguendo tra spese ed entrate. Ogni transazione deve avere importo, data e descrizione. Le spese devono essere associate a una categoria, mentre le entrate devono essere associate a una fonte. Il sistema deve permettere la definizione di budget mensili e per categoria, nonché la gestione di spese ricorrenti tramite un modello astratto. Deve inoltre conservare il percorso dei documenti associati alle spese senza memorizzare i file nel database.

2.2 Analisi delle ambiguità e decisioni progettuali

Termine/Concetto	Criticità/Ambiguità	Decisione progettuale
Periodo temporale	Come associare una spesa a un budget mensile?	Il periodo è derivato dalla data della transazione. L'entità PERIODO serve per la storicizzazione dei budget.
Spesa ricorrente	Generazione dei record nel DB?	È un modello astratto; la logica Java crea nuove SPESE a partire dai parametri del modello.
Privacy/Admin	L'admin vede i dati degli utenti?	Vincolo applicativo: l'admin accede solo a viste aggregate (COUNT/SUM), non ai dati granulari.
Documenti	Dove salvare i file?	Si salva solo il percorso sul file system; il DB non gestisce BLOB per evitare overhead.

2.3 Specifiche ristrutturare

- R1: Gestione delle transazioni finanziarie (spese/entrate) con dettagli minimi obbligatori.
- R2: Classificazione delle spese tramite categorie e classificazione trasversale delle transazioni tramite tag.
- R3: Definizione e storico dei budget mensili e dei budget per categoria.
- R4: Supporto per spese ricorrenti come modello di generazione automatica.
- R5: Salvare solo il percorso dei documenti digitali associati alle spese, senza memorizzare i file nel database.
- R6: Accesso amministrativo limitato a statistiche aggregate, senza visibilità sui dati finanziari personali.

2.4 Glossario dei termini essenziali

- **Transazione:** entità astratta che rappresenta un movimento monetario con ID, data, importo e descrizione.
- **Budget:** reificazione della soglia di spesa legata a Utente, Categoria e Periodo.
- **Tag:** etichetta trasversale associabile a una o più transazioni, utile per analisi granulari non limitate alla sola categoria.
- **Categoria:** classificazione gerarchica 1:N applicata alle spese.
- **Fonte:** origine finanziaria obbligatoria per le entrate.

2.5 Classi di utenza / viste del sistema

- **Utente/Studente:** accede al proprio portafoglio, registra transazioni, gestisce categorie personali e visualizza report.
- **Amministratore:** gestisce categorie/tag di sistema e visualizza statistiche aggregate.

Le viste del sistema includono il portafoglio personale, il cruscotto dei budget e la dashboard degli andamenti finanziari.

2.6 Elenco delle operazioni principali

- Inserimento di una nuova spesa.
- Inserimento di una nuova entrata.
- Creazione/modifica/eliminazione di categorie personali.
- Creazione/modifica/eliminazione di tag personali.
- Definizione e aggiornamento dei budget mensili e per categoria.
- Gestione delle spese ricorrenti.
- Associazione di un documento digitale a una spesa.
- Visualizzazione di report aggregati e statistiche.

2.7 Vincoli espliciti e impliciti

- L'amministratore può consultare solo dati aggregati (COUNT/SUM) e non i dettagli finanziari degli utenti.
- Ogni studente opera sempre sul proprio portafoglio personale: l'email dell'utente autenticato viene usata come chiave di sessione e come riferimento nelle tabelle dipendenti.
- I documenti vengono memorizzati solo tramite il percorso file, non come BLOB nel database.
- Ogni transazione deve contenere data, importo e descrizione.
- Il periodo del budget è derivato dal mese/anno della transazione e viene storicizzato tramite l'entità PERIODO.
- Il budget è opzionale: l'utente può registrare movimenti anche prima di aver definito una soglia di spesa.
- La spesa ricorrente è un modello astratto che genera transazioni effettive, non una transazione reale.

3 Progettazione concettuale

3.1 Criteri generali di modellazione

3.2 Raffinamenti dello schema

3.2.1 Gerarchia delle transazioni (ISA)

Abbiamo una specializzazione totale ed esclusiva tra spese ed entrate, tradotta nello schema relazionale con la relazione TRANSIZIONE e l'attributo **TipoTransazione**. Una spesa è associata obbligatoriamente a una categoria, mentre un'entrata è associata obbligatoriamente a una fonte. I tag sono collegati alle spese tramite la relazione SPESA_TAG, perché servono a rendere più granulare la classificazione delle uscite.

3.2.2 Modellazione ibrida (Sistema vs Personale)

Per CATEGORIA, TAG e FONTE si applica una gerarchia ISA: elementi di sistema creati dall'admin e elementi personali creati dall'utente. Nella traduzione logica questa scelta viene rappresentata con il flag `is_system` e con l'attributo `Email_Proprietario`. Le query applicative caricano gli elementi disponibili con la condizione `is_system = TRUE OR Email_Proprietario`

= ?, in modo da mostrare insieme il catalogo comune e le personalizzazioni dello studente autenticato.

3.2.3 Reificazione del budget

Il budget è un'entità che mette in relazione utente e periodo, con un eventuale riferimento a una categoria. L'assenza della categoria rappresenta un budget mensile complessivo, mentre la presenza della categoria rappresenta un budget specifico. Questa progettazione consente di storicizzare i limiti di spesa e di fissare soglie per ambiti specifici. Nel capitolo di progettazione logica verrà valutata l'introduzione dell'attributo ridondante **Totale_Speso_Attuale** nel budget per migliorare le prestazioni di lettura nella dashboard. Il sistema adotta una politica di budget opzionale: la registrazione di una spesa non richiede necessariamente l'esistenza preventiva di un budget. Se un budget è già presente, il suo **Totale_Speso_Attuale** viene aggiornato nella stessa transazione ACID dell'inserimento della spesa; se invece il budget viene creato in un secondo momento, il valore iniziale dell'attributo ridondante viene calcolato tramite una **SUM()** delle spese pregresse coerenti con periodo, utente ed eventuale categoria.

3.3 Criticità rilevate

- La fonte deve essere legata esclusivamente a ENTRATA; una spesa ha una categoria di uscita.
- La spesa ricorrente punta a CATEGORIA, non a SPESA, perché è un modello generatore di tuple e non una transazione già avvenuta.
- L'utente viene identificato direttamente tramite **Email**, usata come chiave primaria e come chiave esterna nelle tabelle collegate. Questa scelta rende esplicito il legame tra sessione applicativa e isolamento dei dati.

- 3.4 Schema scheletro E/R
- 3.5 Sviluppo delle viste o degli ambiti
 - 3.5.1 Vista Studente e portafoglio personale
 - 3.5.2 Vista classificazione delle transazioni
 - 3.5.3 Vista budget e pianificazione
 - 3.5.4 Vista spese ricorrenti e documenti
 - 3.5.5 Vista amministratore e cataloghi globali
- 3.6 Raffinamenti proposti
- 3.7 Gerarchie presenti nello schema
- 3.8 Associazioni principali e cardinalità
- 3.9 Vincoli non rappresentabili direttamente nello schema E/R
- 3.10 Integrazione delle viste
- 3.11 Schema E/R finale
- 3.12 Dizionario di entità e associazioni

4 Specifiche funzionali

- 4.1 Elenco delle funzionalità richieste
- 4.2 Operazioni di inserimento
- 4.3 Operazioni di modifica
- 4.4 Operazioni di eliminazione
- 4.5 Operazioni di interrogazione
- 4.6 Operazioni statistiche / amministrative
- 4.7 Input, output e tabelle coinvolte per ogni operazione

5 Progettazione logica

- 5.1 Stima del volume dei dati
- 5.2 Descrizione delle operazioni principali e frequenza
- 5.3 Schemi di navigazione
- 5.4 Tabelle degli accessi
- 5.5 Analisi delle ridondanze

L'attributo `Totale_Speso_Attuale` della relazione `BUDGET` è ridondante perché può essere ricavato sommando le spese presenti in `TRANSIZIONE`. La ridondanza viene comunque mantenuta per ridurre il costo delle letture più frequenti della dashboard, dove l'utente controlla spesso lo stato del budget rispetto al limite impostato.

La coerenza viene garantita con due regole applicative:

- quando viene inserita una nuova spesa, l'inserimento della transazione, l'associazione di tag/documento e l'aggiornamento degli eventuali budget esistenti avvengono nella stessa transazione;
- quando un budget viene creato o aggiornato dopo che alcune spese sono già state registrate, il valore iniziale di `Totale_Speso_Attuale` viene calcolato con una query di aggregazione sulle transazioni già presenti.

In questo modo il budget rimane opzionale senza perdere consistenza: l'assenza di un budget non blocca la registrazione della spesa, mentre l'esistenza di un budget impone la sincronizzazione del suo totale ridondante.

Per evitare che la ridondanza diventi una fonte di incoerenza, il progetto affianca al valore memorizzato anche la vista `v_budget_stato`, che calcola lo stato effettivo del budget tramite aggregazione sulle transazioni. Questa vista viene usata come riferimento logico per verificare il valore reale di spesa, residuo e superamento della soglia. Anche lo script di popolamento iniziale calcola `Totale_Speso_Attuale` con una `SUM()` sulle transazioni già inserite, in modo che i dati dimostrativi risultino coerenti anche se osservati direttamente da MySQL Workbench.

5.6 Raffinamento dello schema

5.6.1 Eliminazione delle gerarchie

5.6.2 Soluzione logica anticipata

La specializzazione totale ed esclusiva delle transazioni si traduce in una struttura logica collassata con la relazione `TRANSIZIONE`. La distinzione tra movimenti in uscita e movimenti in entrata è rappresentata da `TipoTransazione`, mentre i vincoli `CHECK` impongono la presenza della categoria solo per le spese e della fonte solo per le entrate.

5.6.3 Eliminazione degli attributi composti

5.6.4 Gestione degli attributi multivalore

5.6.5 Scelta delle chiavi primarie

5.6.6 Traduzione delle associazioni

5.7 Traduzione di entità e associazioni in relazioni

In questa fase si traducono le entità e le associazioni concettuali in relazioni logiche, specificando le chiavi primarie e le chiavi esterne necessarie per mantenere l'integrità referenziale.

5.8 Schema relazionale finale

Lo schema relazionale finale descrive le tabelle definitive, con i rispettivi attributi, tipi di dato e vincoli principali. Questa rappresentazione costituisce la base per la successiva fase di implementazione SQL.

5.9 Vincoli di integrità

In questa sezione si elencano i vincoli di integrità referenziali e di dominio che il database deve rispettare, come le chiavi primarie, le chiavi esterne, i vincoli `UNIQUE` e i controlli sui valori.

5.10 Controllo della normalizzazione

Qui si verifica che lo schema relazionale finale soddisfi i livelli di normalizzazione richiesti, identificando eventuali anomalie di inserimento, aggiornamento o cancellazione.

6 Implementazione SQL

6.1 Script di creazione del database

6.2 Creazione delle tabelle

6.3 Definizione di chiavi primarie e chiavi esterne

6.4 Vincoli NOT NULL, UNIQUE, CHECK

6.5 Trigger, procedure o viste, se usati

6.6 Popolamento iniziale del database

Il popolamento iniziale inserisce utenti demo, periodi, categorie, tag, fonti, transazioni, documenti, budget e spese ricorrenti. Per i budget dimostrativi, il valore ridondante **Totale_Speso_Attuale** non viene lasciato al valore di default, ma viene inizializzato tramite query di aggregazione sulle transazioni già presenti. In questo modo la base dati di prova è coerente sia nella visualizzazione applicativa sia nella consultazione diretta delle tabelle.

6.7 Query SQL delle operazioni principali

6.7.1 Problemi e criticità

Il progetto richiede di garantire l'integrità dei dati durante l'inserimento di una spesa o di un'entrata, e di gestire i budget in modo idempotente. Le principali criticità sono:

- Isolamento utente: ogni query deve filtrare i dati tramite l'email dell'utente autenticato.
- Atomicità: l'inserimento della spesa, l'associazione dei tag, l'eventuale documento e l'aggiornamento dei budget devono avvenire nella stessa transazione.
- Divisione per zero: nelle statistiche percentuali l'assenza di entrate potrebbe causare errori se non si utilizza una protezione adeguata.
- Gestione del budget: la coppia chiave (**Email**, **ID_Periodo**, **ID_Categoria_Key**) deve essere unica per permettere work-flow di upsert senza duplicati.

6.7.2 Analisi

Il sistema utilizza una traduzione collassata della gerarchia: la relazione **TRANSIZIONE** contiene sia spese sia entrate, distinte dall'attributo **TipoTransazione**. Il vincolo **CHECK** impone che una spesa abbia una categoria e nessuna fonte, mentre un'entrata abbia una fonte e nessuna categoria. Per i budget si introduce la logica di "upsert" per evitare duplicati e mantenere coerente l'attributo ridondante **Totale_Speso_Attuale**.

6.7.3 Soluzione SQL

```
1 START TRANSACTION ;
2
3 INSERT INTO PERIODO (Mese , Anno)
4 VALUES (?, ?)
5 ON DUPLICATE KEY UPDATE ID_Periodo = LAST_INSERT_ID(ID_Periodo);
6
7 INSERT INTO TRANSIZIONE
8 (TipoTransazione, Importo, Data, Descrizione, Email, ID_Categoria,
9  ID_Periodo, ID_Fonte)
10 VALUES ('S', ?, ?, ?, ?, ?, LAST_INSERT_ID(), NULL);
11 SET @id_spesa = LAST_INSERT_ID();
12
```

```

13 INSERT INTO SPESA_TAG (ID_Transizione, ID_Tag)
14 VALUES (@id_spesa, ?);
15
16 UPDATE BUDGET
17 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
18 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria IS NULL;
19
20 UPDATE BUDGET
21 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
22 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria = ?;
23
24 COMMIT;

```

Listing 1: Inserimento di una spesa con tag e budget

```

1 SELECT COALESCE(SUM(Importo), 0) AS Totale_Speso
2 FROM TRANSIZIONE
3 WHERE Email = ?
4     AND ID_Periodo = ?
5     AND TipoTransazione = 'S'
6     AND (? IS NULL OR ID_Categoria = ?);
7
8 INSERT INTO BUDGET
9 (Importo_Limite, Alert_Soglia, Totale_Speso_Attuale, ID_Periodo,
10  ID_Categoria, Email)
11 VALUES (?, ?, ?, ?, ?, ?)
12 ON DUPLICATE KEY UPDATE
13 Importo_Limite = VALUES(Importo_Limite),
14 Alert_Soglia = VALUES(Alert_Soglia),
15 Totale_Speso_Attuale = VALUES(Totale_Speso_Attuale);

```

Listing 2: Upsert del budget con totale iniziale coerente

```

1 SELECT
2 (
3     SUM(CASE WHEN TipoTransazione = 'S' THEN Importo ELSE 0 END)
4     /
5     NULLIF(SUM(CASE WHEN TipoTransazione = 'E' THEN Importo ELSE 0
6     END), 0)
7 ) * 100 AS Percentuale_Spese_Su_Entrate
8 FROM TRANSIZIONE
9 WHERE Email = ? AND ID_Periodo = ?;

```

Listing 3: Rapporto Spese/Entrate Mensile

6.7.4 Query di supporto e operazioni amministrative

```

1 SELECT ID_Categoria, Nome, is_system
2 FROM CATEGORIA
3 WHERE is_system = TRUE OR Email_Proprietario = ?
4 ORDER BY is_system DESC, Nome;

```

Listing 4: Categorie, fonti e tag di sistema o personali

```

1 SELECT T.ID_Transizione, T.Importo, T.Data, T.Descrizione,
2        COALESCE(C.Nome, F.Nome) AS Categoria_Fonte,
3        CASE WHEN T.TipoTransazione = 'S' THEN 'Spesa' ELSE 'Entrata' END
4        AS Tipo
5 FROM TRANSIZIONE T
6 LEFT JOIN CATEGORIA C ON T.ID_Categoria = C.ID_Categoria
7 LEFT JOIN FONTE F ON T.ID_Fonte = F.ID_Fonte
8 WHERE T.Email = ? AND T.Data BETWEEN ? AND ?
9 ORDER BY T.Data DESC;

```

Listing 5: Riepilogo transazioni per periodo

```

1 SELECT ID_Budget, Mese, Anno, Ambito,
2        Importo_Limite, Totale_Speso, Residuo, Superato
3 FROM v_budget_stato
4 WHERE Email = ?
5 ORDER BY Anno DESC, Mese DESC, Ambito;

```

Listing 6: Budget con residuo e stato di superamento

6.7.5 Motivazioni

Atomicità applicativa: in Java, l'uso di `conn.setAutoCommit(false)` unito a queste query garantisce che spesa, tag, documento e aggiornamento budget vengano confermati o annullati insieme.

Flessibilità budget: il vincolo `UNIQUE(Email, ID_Periodo, ID_Categoria_Key)` permette alla query `ON DUPLICATE KEY UPDATE` di aggiornare o creare il limite in modo efficiente.

Robustezza matematica: `NULLIF` trasforma lo zero in `NULL`, evitando l'errore di divisione per zero nel database engine.

7 Progettazione dell'applicazione

7.1 Architettura generale dell'applicazione

L'applicazione è organizzata secondo una struttura a livelli ispirata al modello MVC, integrata con il pattern DAO per l'accesso ai dati. Le classi della vista gestiscono l'interfaccia Swing, i controller raccolgono gli eventi e coordinano le richieste, i service contengono la logica applicativa e transazionale, mentre i DAO incapsulano le operazioni effettive sul database tramite JDBC.

Il flusso principale è quindi:

view → controller → service → dao → database

Questa separazione evita che l'interfaccia grafica dipenda direttamente dal database e rende più semplice modificare o testare le singole parti dell'applicazione.

7.2 Tecnologie usate

L'applicazione è sviluppata in Java e utilizza Swing per l'interfaccia grafica. La comunicazione con MySQL avviene tramite JDBC, con connessioni centralizzate nella classe di utilità dedicata. Gli script SQL di creazione, popolamento, viste, trigger e interrogazioni principali sono raccolti nella cartella `doc/sql`, separata dagli schemi concettuali e logici presenti in `doc/schemes`.

7.3 Struttura dei package Java

Il codice sorgente è collocato nel package `it.unibo.sage` ed è suddiviso nei seguenti sottopackage:

- **model**: classi che rappresentano gli oggetti principali del dominio applicativo, come `Utente`, `Transazione`, `Budget`, `Categoria`, `Tag`, `Fonte` e `Documento`;
- **dao**: interfacce DAO e implementazioni JDBC per separare le operazioni logiche sui dati dalla loro realizzazione sul DBMS;
- **service**: logica applicativa, gestione delle transazioni JDBC e coordinamento tra più DAO;
- **controller**: classi che collegano la vista ai service e mantengono stabile l'API usata dall'interfaccia;
- **view**: pannelli, frame e componenti grafici Java Swing;
- **utils**: classi di supporto, tra cui la gestione della connessione al database.

7.4 Collegamento al database tramite JDBC

La classe `DatabaseConnection` centralizza la creazione delle connessioni JDBC. I service aprono le connessioni necessarie e, quando una funzionalità coinvolge più tabelle, disattivano l'autocommit per garantire atomicità. Ad esempio, l'inserimento di una spesa, l'associazione dei tag, l'eventuale salvataggio del documento e l'aggiornamento dei budget avvengono nella stessa transazione applicativa.

Per la schermata dei budget, il service ricalcola lo stato di avanzamento partendo dalle transazioni effettive, così la vista applicativa resta allineata ai movimenti registrati anche quando un budget viene creato dopo l'inserimento di alcune spese.

7.5 Organizzazione delle classi principali

Le classi del package `model` rappresentano gli oggetti principali del dominio applicativo e vengono utilizzate per trasferire i dati tra DAO, service, controller e interfaccia grafica. La classe `Documento` rappresenta un file associato a una transazione, come ricevuta, fattura o prova di pagamento; nel database viene memorizzato il percorso del file e non il contenuto binario.

Per l'accesso ai dati è stato adottato il pattern DAO, separando le operazioni logiche sulle entità dalla loro implementazione concreta tramite JDBC. In questo modo i controller e i service non dipendono da query sparse nell'interfaccia, ma comunicano con classi dedicate alla persistenza.

7.6 Gestione degli errori e validazione input

Gli errori SQL vengono propagati verso i controller e gestiti dalla vista con messaggi all'utente. Le operazioni composte usano rollback in caso di eccezione, così da evitare stati parziali del database. Le validazioni di base, come campi obbligatori e importi non validi, sono gestite prima dell'invio dei dati ai service.

7.7 Interfaccia grafica Java Swing

L'interfaccia grafica è realizzata con Java Swing e organizzata in pannelli dedicati alle funzionalità principali: autenticazione, dashboard, movimenti, budget e navigazione laterale. I componenti grafici non eseguono query direttamente, ma invocano i controller, mantenendo separata la logica di presentazione dalla logica applicativa.

7.8 Schermate principali

Le schermate principali dell'applicazione sono raccolte nella cartella `doc/screenshots` e documentano i flussi più importanti: login, dashboard, registrazione dei movimenti, gestione dei budget e spese ricorrenti.

7.9 Flussi d'uso principali

Registrazione studente
Inserimento di una spesa
Inserimento di una entrata
Definizione di un budget
Gestione di una spesa ricorrente
Associazione di un documento digitale
Visualizzazione statistiche amministratore

8 Test e validazione

8.1 Dati di test utilizzati

8.2 Test delle query SQL

8.3 Test delle funzionalità applicative

8.4 Casi limite gestiti

8.5 Errori noti o limitazioni

9 Conclusioni

9.1 Risultato ottenuto

9.2 Scelte progettuali principali

9.3 Possibili sviluppi futuri

- A Script SQL completo
- B Query SQL complete
- C Screenshot dell'applicazione
- D Manuale utente sintetico
- E Eventuale link al repository